A FENICS GUIDE

Introduction

FEniCS is an open-source computing platform for solving partial differential equations (PDEs) using the finite element method (FEM). FEniCS enables users to quickly translate scientific models into efficient finite element code. With the high-level Python and C++ interfaces to FEniCS, it is easy to get started, but FEniCS offers also powerful capabilities for more experienced programmers. FEniCS runs on a multitude of platforms ranging from laptops to high-performance computers.

Date: May 2026.

1. Installation

There are two methods for installation of FEniCS ; one is using a WSL(Windows Subsystem for Linux) and the other is using a container, for example-Docker.
Here we will install FEniCS using WSL –
A step by step guide :-

Part 1 - Installation of WSL.

(1) Run Powershell as Administrator. Give any neccessary permissions.
(2) Type :-

```
wsl --install
```

then restart pc... Open Powershell again
Type :-

```
wsl -l -o
```

It shows all versions of Linux available to download. Choose and download your required linux version using following example-

```
wsl.exe --install -d Ubuntu-22.04
```

(3) Set Unix username and password.
(4) After completion, close Powershell. We will now work using terminal.

Part 2 - Installation of Conda. Conda is an open-source, cross-platform package and environment manager designed to install, run, and update software packages and their dependencies. It excels at creating isolated environments for different projects, making it a popular tool in data science, machine learning, and Python/R development to prevent package version conflicts.

(1) Open terminal. Type-

```
wsl
```

Wait. Linux subsystem will open.
(2) In Google, search - "Miniconda install Linux"
(3) Navigate through the Anaconda website. You have to copy paste some commands. Go to Linux installer tab.
(4) Go to download section ->Linux x86 ->Copy the code. The code somewhat looks like this->

```
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

(5) Install Conda using installation code. The code looks like this ->

```
bash Miniconda3-latest-Linux-x86_64.sh
```

(6) Accept all terms and conditions(if any) and activate and initialize Miniconda(Follow the on screen instructions)
(7) For Miniconda to take effect, close and re open terminal , then type `wsl` again.

Part 3 - Installation of FEniCS.

(1) Go to FEniCS website and copy the installation code. Looks like this->

```
conda create -n fenicsx-env
conda activate fenicsx-env
conda install -c conda-forge fenics-dolfinx mpich pyvista
```

Convert all "env" to "real". Should look lke this-

```
conda create -n fenicsx-real
conda activate fenicsx-real
conda install -c conda-forge fenics-dolfinx mpich pyvista
```

Accept all terms and conditions and install.

(2) Similarly make a "complex" environment.
(3) FEniCS Installation is complete.

Note:- To activate, type - `conda activate fenicsx-real`
To deactivate, type - `conda deactivate`

Part 4 - Setting up VS Code.

(1) Keep terminal in the background with WSL running.
(2) Open VS Code
(3) Go to Extensions tab. It is on the left side of the screen.
(4) Search for WSL extension and install it.
(5) A new tab - Remote explorer - will appear. Click on it. Click on Ubuntu. Then on "Connect in current Window".

Part 5 - Testing FEniCS using an example.

(1) Click on Open Folder, then navigate to /mnt/c/Users/Username(not Unix one, but your windows one)
(2) Create new file - test.py
(3) Copy and Paste the general Poisson Equation code from Jorgen Dokken's github repository ->

```
---------------------------------------------------------------------------
from mpi4py import MPI
from dolfinx import mesh
import numpy

domain = mesh.create_unit_square(MPI.COMM_WORLD, 8, 8, mesh.CellType.quadrilateral)

from dolfinx import fem

V = fem.functionspace(domain, ("Lagrange", 1))

uD = fem.Function(V)
uD.interpolate(lambda x: 1 + x[0] ** 2 + 2 * x[1] ** 2)
```

```
tdim = domain.topology.dim
fdim = tdim - 1
domain.topology.create_connectivity(fdim, tdim)
boundary_facets = mesh.exterior_facet_indices(domain.topology)

boundary_dofs = fem.locate_dofs_topological(V, fdim, boundary_facets)
bc = fem.dirichletbc(uD, boundary_dofs)

import ufl

u = ufl.TrialFunction(V)
v = ufl.TestFunction(V)

from dolfinx import default_scalar_type

f = fem.Constant(domain, default_scalar_type(-6))

a = ufl.dot(ufl.grad(u), ufl.grad(v)) * ufl.dx
L = f * v * ufl.dx

from dolfinx.fem.petsc import LinearProblem

problem = LinearProblem(
    a,
    L,
    bcs=[bc],
    petsc_options={"ksp_type": "preonly", "pc_type": "lu"},
    petsc_options_prefix="Poisson",
)
uh = problem.solve()

V2 = fem.functionspace(domain, ("Lagrange", 2))
uex = fem.Function(V2, name="u_exact")
uex.interpolate(lambda x: 1 + x[0] ** 2 + 2 * x[1] ** 2)

L2_error = fem.form(ufl.inner(uh - uex, uh - uex) * ufl.dx)
error_local = fem.assemble_scalar(L2_error)
error_L2 = numpy.sqrt(domain.comm.allreduce(error_local, op=MPI.SUM))

error_max = numpy.max(numpy.abs(uD.x.array - uh.x.array))
if domain.comm.rank == 0:  # Only print the error on one process
    print(f"Error_L2 : {error_L2:.2e}")
    print(f"Error_max : {error_max:.2e}")

import pyvista
```

```
print(pyvista.global_theme.jupyter_backend)

from dolfinx import plot

domain.topology.create_connectivity(tdim, tdim)
topology, cell_types, geometry = plot.vtk_mesh(domain, tdim)
grid = pyvista.UnstructuredGrid(topology, cell_types, geometry)

plotter = pyvista.Plotter()
plotter.add_mesh(grid, show_edges=True)
plotter.view_xy()
if not pyvista.OFF_SCREEN:
    plotter.show()
else:
    figure = plotter.screenshot("fundamentals_mesh.png")

u_topology, u_cell_types, u_geometry = plot.vtk_mesh(V)

u_grid = pyvista.UnstructuredGrid(u_topology, u_cell_types, u_geometry)
u_grid.point_data["u"] = uh.x.array.real
u_grid.set_active_scalars("u")
u_plotter = pyvista.Plotter()
u_plotter.add_mesh(u_grid, show_edges=True)
u_plotter.view_xy()
if not pyvista.OFF_SCREEN:
    u_plotter.show()

warped = u_grid.warp_by_scalar()
plotter2 = pyvista.Plotter()
plotter2.add_mesh(warped, show_edges=True, show_scalar_bar=True)
if not pyvista.OFF_SCREEN:
    plotter2.show()

from dolfinx import io
from pathlib import Path

results_folder = Path("results")
results_folder.mkdir(exist_ok=True, parents=True)
filename = results_folder / "fundamentals"
with io.VTXWriter(domain.comm, filename.with_suffix(".bp"), [uh]) as vtx:
    vtx.write(0.0)
with io.XDMFFile(domain.comm, filename.with_suffix(".xdmf"), "w") as xdmf:
    xdmf.write_mesh(domain)
    xdmf.write_function(uh)
```

---

(4) Open New Terminal.
(5) If the environment shows `fenicsx-real` , then fine, or else type- `conda activate fenicsx-real`
(6) Type in ->`python3 test.py`
(7) The program must run.

Note:- To turn off WSL environment in VS Code-

(1) Click on lower left corner - "WSL:Ubuntu...."
(2) Click on - Close Remote Connection

2. **Weak Formulation of PDEs-**

Weak formulation of Partial Differential Equations involves multipling the PDE with a Test Function - $v(x)$ and integrating over a given domain. For example the Poisson equation in 1-D is given as -

$$\frac{\partial^2 u(x)}{\partial x^2} = f(x) \tag{1}$$

where $u(x)$ is a field and $f(x)$ is the source term. Now multiply both sides with an arbitrary test function - $v(x)$, and integrate it -

$$\int \frac{\partial^2 u(x)}{\partial x^2} v(x) dx = \int f(x) v(x) dx \tag{2}$$

Since the Test Function is arbitrary, to narrow down to the solution, we introduce Boundary Conditions and Initial Conditions. Boundary Conditions are for Space coordinates and Initial Conditions are for the Time coordinate. Boundary conditions can be Dirichlet type, Neumann type, Mixed type, etc.

Dirichlet type is the condition in which boundary value are constant. In Neumann, boundary values are fluxes.

The Boundary conditions can further be Homogeneous (0 at boundary) or Non-Homogeneous(Non zero at boundary).

3. **2D Transient Heat Diffusion Equation -**

The General Transport equation is given by -

$$\frac{\partial T}{\partial t} + u \cdot \nabla T = \alpha \nabla^2 T + f \tag{3}$$

According to our problem, Convective term is zero. Therefore it turns out to be-

$$\frac{\partial T}{\partial t} = \alpha \nabla^2 T + f \tag{4}$$

Take $\alpha = 1$. Rewrite in dot product form -

$$\frac{\partial T}{\partial t} = \nabla \cdot \nabla T + f \tag{5}$$

Multiply v (test function) on both sides -

$$\frac{\partial T}{\partial t} v = \nabla v \cdot \nabla T + fv \tag{6}$$

Here T = u. Refer to the section - Weak Formulation of PDEs. Discretize the equation. Next u is taken u_n and previous value is u. Time difference is taken as dt

$$\frac{u_n - u}{dt} v = \nabla v \cdot \nabla T + fv \tag{7}$$

$$u_n v - uv = \nabla v \cdot \nabla T dt + fv dt \tag{8}$$

Integrate both sides wrt to x -

$$\int u_n v dx - \int uv dx = \int \nabla v \cdot \nabla T dt dx + \int fv dt dx \tag{9}$$

The FEniCS code is as follows-

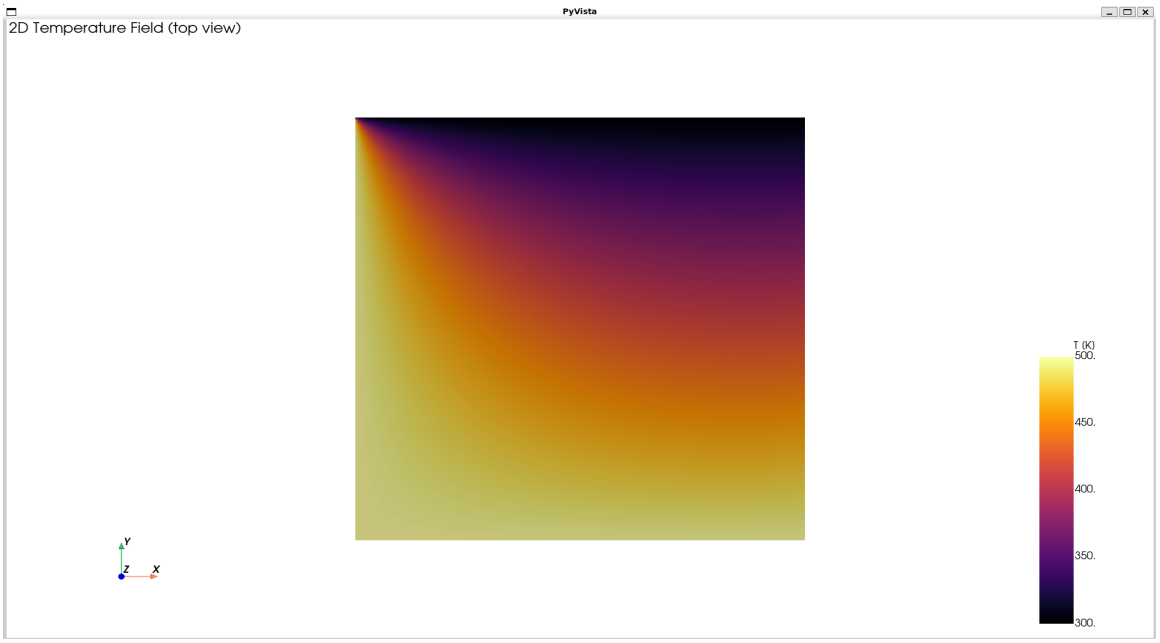
```
-------------------------------------------------------------------------------------------
"""
2D Transient Heat Conduction | FEniCSx / DOLFINx
================================================
Governing PDE (strong form):
    rho*cp * dT/dt = div(k * grad(T)) + Q      in Omega = [0,1] x [0,1]

Boundary conditions (all four walls distinct):
    LEFT   (x=0): Dirichlet  T = T_cold = 300 K        (cold sink)
    RIGHT  (x=1): Neumann    k*dT/dn = +q_right         (heat flux IN)
    BOTTOM (y=0): Dirichlet  T = T_hot  = 500 K         (hot base)
    TOP    (y=1): Neumann    k*dT/dn = -q_top           (heat flux OUT / convective loss)

Physics expectation:
    - Strong gradient bottom (hot) -> top (cooler)
    - Left wall anchored hot, right wall receiving flux
    - A 2-D "saddle" or skewed temperature field | NOT uniform
"""

import numpy as np
```

```
from mpi4py import MPI
from petsc4py import PETSc
from dolfinx import mesh, fem, io, plot
from dolfinx.fem import functionspace, Function, Constant, dirichletbc
from dolfinx.fem.petsc import LinearProblem
import ufl
from ufl import TestFunction, TrialFunction, dx, ds, inner, grad
import pyvista

# 1. Material & source parameters  (steel-like)
k_val    = 0.60    # Thermal conductivity            [W/m·K]
rho_val  = 1000.0  # Density                         [kg/m³]
cp_val   = 4184.0  # Specific heat                   [J/kg·K]
Q_val    = 0.0     # Volumetric source               [W/m³]
q_right  = 10.0    # Inward flux, right BC           [W/m²]
q_top    = 0.0     # Outward flux, top BC            [W/m²]
T_cold   = 300.0   # Right wall and Top wall temp    [K]
T_hot    = 500.0   # Left Wall and Bottom wall temp  [K]

# Thermal diffusivity & timescale
alpha = k_val / (rho_val * cp_val)
tau   = 10**2 / alpha           # diffusion timescale [s]
t_end = 2.5 * tau
dt    = tau / 20
n_steps = int(t_end / dt)

# 2. Mesh  (refined 128x128 for smooth 2-D field)
comm   = MPI.COMM_WORLD
domain = mesh.create_unit_square(comm, 128, 128, mesh.CellType.quadrilateral)
V      = functionspace(domain, ("Lagrange", 1))

T   = TrialFunction(V)
v   = TestFunction(V)
T_n = Function(V)      # previous step
T_h = Function(V)      # current solution

# Initialise field: linear interpolation between T_cold and T_hot
T_n.interpolate(lambda x: T_cold + (T_hot - T_cold) * x[1] + (((T_hot-T_cold)**2)/2)*(x[1]**2))
T_h.x.array[:] = T_n.x.array

# UFL constants
k   = Constant(domain, PETSc.ScalarType(k_val))
rho = Constant(domain, PETSc.ScalarType(rho_val))
cp  = Constant(domain, PETSc.ScalarType(cp_val))
Q   = Constant(domain, PETSc.ScalarType(Q_val))
qR  = Constant(domain, PETSc.ScalarType(q_right))
```

```
qT  = Constant(domain, PETSc.ScalarType(q_top))
Dt  = Constant(domain, PETSc.ScalarType(dt))

# 3. Boundary facet tags
fdim = domain.topology.dim - 1

def mark_facets(domain, fdim):
    left_f   = mesh.locate_entities_boundary(domain, fdim, lambda x: np.isclose(x[0], 0.0))
    right_f  = mesh.locate_entities_boundary(domain, fdim, lambda x: np.isclose(x[0], 1.0))
    bottom_f = mesh.locate_entities_boundary(domain, fdim, lambda x: np.isclose(x[1], 0.0))
    top_f    = mesh.locate_entities_boundary(domain, fdim, lambda x: np.isclose(x[1], 1.0))

    facets  = np.concatenate([left_f, right_f, bottom_f, top_f])
    markers = np.concatenate([
        np.full_like(left_f,   1, dtype=np.int32),
        np.full_like(right_f,  2, dtype=np.int32),
        np.full_like(bottom_f, 3, dtype=np.int32),
        np.full_like(top_f,    4, dtype=np.int32),
    ])
    # sort by facet index (required by DOLFINx)
    order   = np.argsort(facets)
    return mesh.meshtags(domain, fdim, facets[order], markers[order])

facet_tags = mark_facets(domain, fdim)
ds_tagged  = ufl.Measure("ds", domain=domain, subdomain_data=facet_tags)

# 4. Dirichlet BCs -- -- -- -- --
# LEFT T = T_hot
left_dofs = fem.locate_dofs_geometrical(V, lambda x: np.isclose(x[0], 0.0))
bc_left   = dirichletbc(Constant(domain, PETSc.ScalarType(T_hot)), left_dofs, V)

# BOTTOM T=T_hot
bot_dofs  = fem.locate_dofs_geometrical(V, lambda x: np.isclose(x[1], 0.0))
bc_bot    = dirichletbc(Constant(domain, PETSc.ScalarType(T_hot)),  bot_dofs,  V)

# TOP -
top_dofs  = fem.locate_dofs_geometrical(V, lambda x: np.isclose(x[1], 1.0))
bc_top    = dirichletbc(Constant(domain, PETSc.ScalarType(T_cold)),  top_dofs,  V)

bcs = [bc_left, bc_bot, bc_top]

# 5. Variational form  (Backward Euler)
#
#  rho*cp*(T - T_n)/dt * v dx
#  + k * grad(T)·grad(v) dx
#  = rho*cp*T_n/dt * v dx
```

```
#  + Q * v dx
#  + q_right * v ds(right)     [+: flux INTO domain]
#  - q_top   * v ds(top)       [-: flux OUT of domain]
a = (rho * cp / Dt) * inner(T, v) * dx  + k * inner(grad(T), grad(v)) * dx

L = (rho * cp / Dt) * inner(T_n, v) * dx + inner(Q, v) * dx + inner(qR, v) * ds_tagged(2)
                                                    - inner(qT, v) * ds_tagged(4)

# 6. Linear solver
problem = LinearProblem(
    a, L,
    bcs=bcs,
    u=T_h,
    petsc_options_prefix="heat_",
    petsc_options={
        "ksp_type":      "cg",
        "pc_type":       "hypre",
        "pc_hypre_type": "boomeramg",
    }
)

# 7. Time-stepping
vtx = io.VTXWriter(comm, "heat2d_solution.bp", [T_h], engine="BP4")
vtx.write(0.0)
t = 0.0
for step in range(n_steps):
    t += dt
    problem.solve()
    T_n.x.array[:] = T_h.x.array
    vtx.write(t)
    arr = T_h.x.array
vtx.close()

# Pyvista visualizaation -
topology, cell_types, geometry = plot.vtk_mesh(V)
grid = pyvista.UnstructuredGrid(topology, cell_types, geometry)
T_array = T_h.x.array.real
grid.point_data["Temperature [K]"] = T_array
plotter = pyvista.Plotter()
plotter.subplot(0, 0)
plotter.add_text("2D Temperature Field (top view)", font_size=10)
flat = grid.copy()
flat.set_active_scalars("Temperature [K]")
plotter.add_mesh(flat, show_edges=False, cmap="inferno",
   scalar_bar_args={"title": "T [K]", "vertical": True})
plotter.view_xy()
```

```
plotter.add_axes()
plotter.show()
```



4. **2D Hagen-Poiseuille Flow -**

Boundary Condtions -

- Upper Wall and Bottom Wall - No Slip boundary condition.
- Left (Entry) - Pressure is 8 units
- Right (Exit) - Pressure is 0 units

The Fenics Code is as follows -

```
---------------------------------------------------------------------------------------
from mpi4py import MPI
from petsc4py import PETSc
import numpy as np
import pyvista

from dolfinx.fem import (
    Constant,
    Function,
    extract_function_spaces,
    functionspace,
    assemble_scalar,
    dirichletbc,
    form,
    locate_dofs_geometrical,
)
from dolfinx.fem.petsc import (
    assemble_matrix,
    assemble_vector,
    apply_lifting,
    create_vector,
    set_bc,
)
from dolfinx.io import VTXWriter
from dolfinx.mesh import create_unit_square
from dolfinx.plot import vtk_mesh
from basix.ufl import element
from ufl import (
    FacetNormal,
    Identity,
    TestFunction,
    TrialFunction,
    div,
    dot,
    ds,
    dx,
    inner,
    lhs,
    nabla_grad,
```

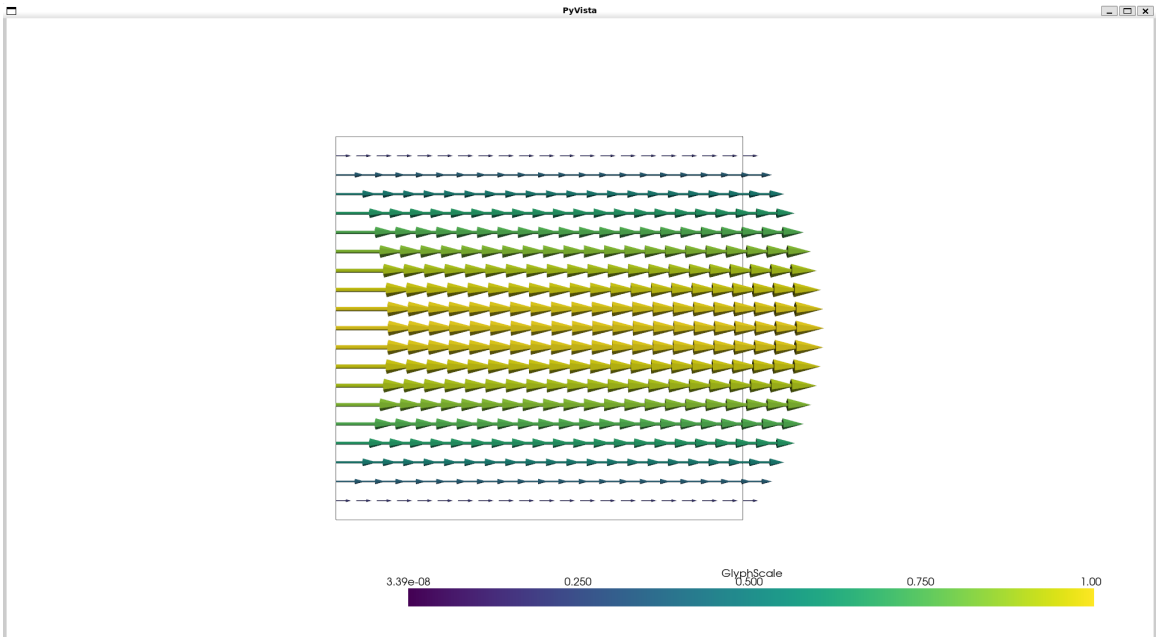
```
    rhs,
    sym,
)
from pathlib import Path
from dolfinx import mesh

# Define the time parameters -
t = 0.0
T = 10.0
num_steps = 500
dt = T / num_steps

# Define the mesh -
mesh = create_unit_square(MPI.COMM_WORLD, 10, 10, mesh.CellType.quadrilateral)

# Create function spaces -
v_cg2 = element("Lagrange", mesh.basix_cell(), 2, shape=(mesh.geometry.dim,))
s_cg1 = element("Lagrange", mesh.basix_cell(), 1)
V = functionspace(mesh, v_cg2)
Q = functionspace(mesh, s_cg1)

# Define the trial and test functions -
u = TrialFunction(V)
v = TestFunction(V)
p = TrialFunction(Q)
q = TestFunction(Q)

# Set the Boundary Conditions -
def walls(x):
    return np.logical_or(np.isclose(x[1], 0), np.isclose(x[1], 1))
wall_dofs = locate_dofs_geometrical(V, walls)
u_noslip = np.array((0,) * mesh.geometry.dim, dtype=PETSc.ScalarType)
bc_noslip = dirichletbc(u_noslip, wall_dofs, V)

def inflow(x):
    return np.isclose(x[0], 0)
inflow_dofs = locate_dofs_geometrical(Q, inflow)
bc_inflow = dirichletbc(PETSc.ScalarType(8), inflow_dofs, Q)

def outflow(x):
    return np.isclose(x[0], 1)
outflow_dofs = locate_dofs_geometrical(Q, outflow)
bc_outflow = dirichletbc(PETSc.ScalarType(0), outflow_dofs, Q)
bcu = [bc_noslip]
bcp = [bc_inflow, bc_outflow]
```

```
# Define the other constant parameters -
u_n = Function(V)
u_n.name = "u_n"
U = 0.5 * (u_n + u)
n = FacetNormal(mesh)
f = Constant(mesh, PETSc.ScalarType((0, 0)))
k = Constant(mesh, PETSc.ScalarType(dt))
mu = Constant(mesh, PETSc.ScalarType(1))
rho = Constant(mesh, PETSc.ScalarType(1))

# Define the Strain rate Tensor and the Stress Tensor -
def epsilon(u):
    return sym(nabla_grad(u))

def sigma(u, p):
    return 2 * mu * epsilon(u) - p * Identity(len(u))

# Define variational problem for the first step -
p_n = Function(Q)
p_n.name = "p_n"
F1 = rho * dot((u - u_n) / k, v) * dx
F1 += rho * dot(dot(u_n, nabla_grad(u_n)), v) * dx
F1 += inner(sigma(U, p_n), epsilon(v)) * dx
F1 += dot(p_n * n, v) * ds - dot(mu * nabla_grad(U) * n, v) * ds
F1 -= dot(f, v) * dx
a1 = form(lhs(F1))
L1 = form(rhs(F1))
A1 = assemble_matrix(a1, bcs=bcu)
A1.assemble()
b1 = create_vector(extract_function_spaces(L1))

# Define variational problem for step 2 -
u_ = Function(V)
a2 = form(dot(nabla_grad(p), nabla_grad(q)) * dx)
L2 = form(dot(nabla_grad(p_n), nabla_grad(q)) * dx - (rho / k) * div(u_) * q * dx)
A2 = assemble_matrix(a2, bcs=bcp)
A2.assemble()
b2 = create_vector(extract_function_spaces(L2))

# Define variational problem for step 3 -
p_ = Function(Q)
a3 = form(rho * dot(u, v) * dx)
L3 = form(rho * dot(u_, v) * dx - k * dot(nabla_grad(p_ - p_n), v) * dx)
A3 = assemble_matrix(a3)
A3.assemble()
b3 = create_vector(extract_function_spaces(L3))
```

```
# Create the solvers -
# Solver for step 1
solver1 = PETSc.KSP().create(mesh.comm)
solver1.setOperators(A1)
solver1.setType(PETSc.KSP.Type.BCGS)
pc1 = solver1.getPC()
pc1.setType(PETSc.PC.Type.HYPRE)
pc1.setHYPREType("boomeramg")
# Solver for step 2
solver2 = PETSc.KSP().create(mesh.comm)
solver2.setOperators(A2)
solver2.setType(PETSc.KSP.Type.BCGS)
pc2 = solver2.getPC()
pc2.setType(PETSc.PC.Type.HYPRE)
pc2.setHYPREType("boomeramg")
# Solver for step 3
solver3 = PETSc.KSP().create(mesh.comm)
solver3.setOperators(A3)
solver3.setType(PETSc.KSP.Type.CG)
pc3 = solver3.getPC()
pc3.setType(PETSc.PC.Type.SOR)

def u_exact(x):
    values = np.zeros((2, x.shape[1]), dtype=PETSc.ScalarType)
    values[0] = 4 * x[1] * (1.0 - x[1])
    return values

u_ex = Function(V)
u_ex.interpolate(u_exact)

L2_error = form(dot(u_ - u_ex, u_ - u_ex) * dx)

# Time loop
for i in range(num_steps):
    # Update current time step
    t += dt

    # Step 1: Tentative veolcity step
    with b1.localForm() as loc_1:
        loc_1.set(0)
    assemble_vector(b1, L1)
    apply_lifting(b1, [a1], [bcu])
    b1.ghostUpdate(addv=PETSc.InsertMode.ADD_VALUES, mode=PETSc.ScatterMode.REVERSE)
    set_bc(b1, bcu)
    solver1.solve(b1, u_.x.petsc_vec)
```

```
    u_.x.scatter_forward()

    # Step 2: Pressure corrrection step
    with b2.localForm() as loc_2:
        loc_2.set(0)
    assemble_vector(b2, L2)
    apply_lifting(b2, [a2], [bcp])
    b2.ghostUpdate(addv=PETSc.InsertMode.ADD_VALUES, mode=PETSc.ScatterMode.REVERSE)
    set_bc(b2, bcp)
    solver2.solve(b2, p_.x.petsc_vec)
    p_.x.scatter_forward()

    # Step 3: Velocity correction step
    with b3.localForm() as loc_3:
        loc_3.set(0)
    assemble_vector(b3, L3)
    b3.ghostUpdate(addv=PETSc.InsertMode.ADD_VALUES, mode=PETSc.ScatterMode.REVERSE)
    solver3.solve(b3, u_.x.petsc_vec)
    u_.x.scatter_forward()
    # Update variable with solution form this time step
    u_n.x.array[:] = u_.x.array[:]
    p_n.x.array[:] = p_.x.array[:]

    # Compute error at current time-step
    error_L2 = np.sqrt(mesh.comm.allreduce(assemble_scalar(L2_error), op=MPI.SUM))
    error_max = mesh.comm.allreduce(
        np.max(u_.x.petsc_vec.array - u_ex.x.petsc_vec.array), op=MPI.MAX
    )

# Close xmdf file
b1.destroy()
b2.destroy()
b3.destroy()
solver1.destroy()
solver2.destroy()
solver3.destroy()

# Visualization using vectors -
topology, cell_types, geometry = vtk_mesh(V)
values = np.zeros((geometry.shape[0], 3), dtype=np.float64)
values[:, : len(u_n)] = u_n.x.array.real.reshape((geometry.shape[0], len(u_n)))

# Create a point cloud of glyphs
function_grid = pyvista.UnstructuredGrid(topology, cell_types, geometry)
function_grid["u"] = values
glyphs = function_grid.glyph(orient="u", factor=0.2)
```

```
# Create a pyvista-grid for the mesh
tdim = mesh.topology.dim
mesh.topology.create_connectivity(tdim, tdim)
grid = pyvista.UnstructuredGrid(*vtk_mesh(mesh, tdim))

# Create plotter
plotter = pyvista.Plotter()
plotter.add_mesh(grid.outline(), style="wireframe", color="k")
plotter.add_mesh(glyphs)
plotter.view_xy()

if not pyvista.OFF_SCREEN:
    plotter.show()
else:
    fig_as_array = plotter.screenshot("glyphs.png")
```



5. **2D Couette Flow -**

Boundary Condtions -

- Upper Wall - Maximum Velocity U_max
- Bottom Wall - No slip boundary condition

The Fenics Code is as follows -

```
from mpi4py import MPI
from petsc4py import PETSc
import numpy as np
import pyvista

from dolfinx.fem import (
    Constant,
    Function,
    extract_function_spaces,
    functionspace,
    assemble_scalar,
    dirichletbc,
    form,
    locate_dofs_geometrical,
)
from dolfinx.fem.petsc import (
    assemble_matrix,
    assemble_vector,
    apply_lifting,
    create_vector,
    set_bc,
)
from dolfinx.io import VTXWriter
from dolfinx.mesh import create_unit_square
from dolfinx.plot import vtk_mesh
from basix.ufl import element
from ufl import (
    FacetNormal,
    Identity,
    TestFunction,
    TrialFunction,
    div,
    dot,
    ds,
    dx,
    inner,
    lhs,
    nabla_grad,
    rhs,
```

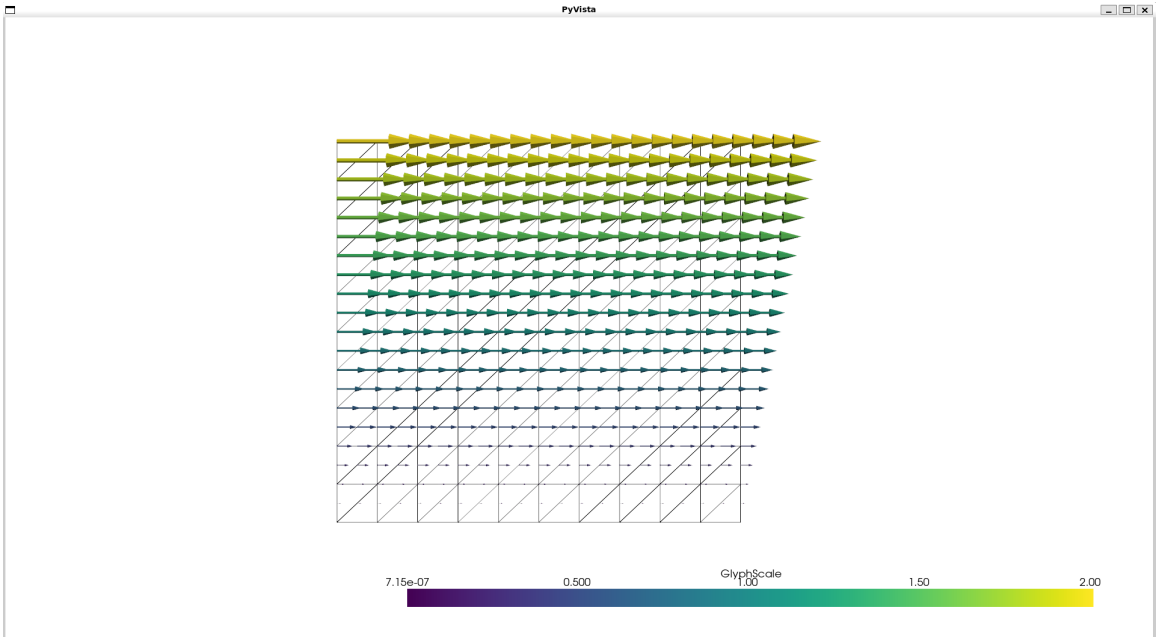
```
    sym,
)

# 1. Mesh and Time-stepping
mesh = create_unit_square(MPI.COMM_WORLD, 10, 10)
t = 0.0
T = 10.0
num_steps = 500
dt = T / num_steps

# 2. Function Spaces (Taylor-Hood Elements)
v_cg2 = element("Lagrange", mesh.basix_cell(), 2, shape=(mesh.geometry.dim,))
s_cg1 = element("Lagrange", mesh.basix_cell(), 1)
V = functionspace(mesh, v_cg2)
Q = functionspace(mesh, s_cg1)

u = TrialFunction(V)
v = TestFunction(V)
p = TrialFunction(Q)
q = TestFunction(Q)

# 3. Boundary Conditions for Couette Flow
# Bottom Wall: Stationary (No-slip)
def bottom_wall(x):
    return np.isclose(x[1], 0)

bottom_wall_dofs = locate_dofs_geometrical(V, bottom_wall)
u_noslip = np.array((0.0, 0.0), dtype=PETSc.ScalarType)
bc_bottom = dirichletbc(u_noslip, bottom_wall_dofs, V)

# Top Wall: Moving to the right
def top_wall(x):
    return np.isclose(x[1], 1)

top_wall_dofs = locate_dofs_geometrical(V, top_wall)
u_moving = np.array((1.0, 0.0), dtype=PETSc.ScalarType) # U = 1.0
bc_top = dirichletbc(u_moving, top_wall_dofs, V)

# Combine velocity BCs
bcu = [bc_bottom, bc_top]

# Left and Right Boundaries: Zero pressure gradient
def left_right(x):
    return np.logical_or(np.isclose(x[0], 0), np.isclose(x[0], 1))

lr_dofs = locate_dofs_geometrical(Q, left_right)
```

```
bc_p_zero = dirichletbc(PETSc.ScalarType(0.0), lr_dofs, Q)

# Combine pressure BCs
bcp = [bc_p_zero]

# 4. Variational Formulation
u_n = Function(V)
u_n.name = "u_n"
U = 0.5 * (u_n + u)
n = FacetNormal(mesh)
f = Constant(mesh, PETSc.ScalarType((0, 0)))
k = Constant(mesh, PETSc.ScalarType(dt))
mu = Constant(mesh, PETSc.ScalarType(1))
rho = Constant(mesh, PETSc.ScalarType(1))

def epsilon(u):
    """Strain-rate tensor."""
    return sym(nabla_grad(u))

def sigma(u, p):
    """Stress tensor."""
    return 2 * mu * epsilon(u) - p * Identity(mesh.geometry.dim)

p_n = Function(Q)
p_n.name = "p_n"

# Step 1: Tentative velocity
F1 = rho * dot((u - u_n) / k, v) * dx
F1 += rho * dot(dot(u_n, nabla_grad(u_n)), v) * dx
F1 += inner(sigma(U, p_n), epsilon(v)) * dx
F1 += dot(p_n * n, v) * ds - dot(mu * nabla_grad(U) * n, v) * ds
F1 -= dot(f, v) * dx
a1 = form(lhs(F1))
L1 = form(rhs(F1))
A1 = assemble_matrix(a1, bcs=bcu)
A1.assemble()
b1 = create_vector(extract_function_spaces(L1))

# Step 2: Pressure correction
u_ = Function(V)
a2 = form(dot(nabla_grad(p), nabla_grad(q)) * dx)
L2 = form(dot(nabla_grad(p_n), nabla_grad(q)) * dx - (rho / k) * div(u_) * q * dx)
A2 = assemble_matrix(a2, bcs=bcp)
A2.assemble()
b2 = create_vector(extract_function_spaces(L2))
```

```
# Step 3: Velocity correction
p_ = Function(Q)
a3 = form(rho * dot(u, v) * dx)
L3 = form(rho * dot(u_, v) * dx - k * dot(nabla_grad(p_ - p_n), v) * dx)
A3 = assemble_matrix(a3)
A3.assemble()
b3 = create_vector(extract_function_spaces(L3))

# 5. Solvers
solver1 = PETSc.KSP().create(mesh.comm)
solver1.setOperators(A1)
solver1.setType(PETSc.KSP.Type.BCGS)
pc1 = solver1.getPC()
pc1.setType(PETSc.PC.Type.HYPRE)
pc1.setHYPREType("boomeramg")

solver2 = PETSc.KSP().create(mesh.comm)
solver2.setOperators(A2)
solver2.setType(PETSc.KSP.Type.CG)
pc2 = solver2.getPC()
pc2.setType(PETSc.PC.Type.HYPRE)
pc2.setHYPREType("boomeramg")

solver3 = PETSc.KSP().create(mesh.comm)
solver3.setOperators(A3)
solver3.setType(PETSc.KSP.Type.CG)
pc3 = solver3.getPC()
pc3.setType(PETSc.PC.Type.SOR)

# 8. Time-stepping loop
for i in range(num_steps):
    t += dt

    # Step 1: Tentative velocity step
    with b1.localForm() as loc_1:
        loc_1.set(0)
    assemble_vector(b1, L1)
    apply_lifting(b1, [a1], [bcu])
    b1.ghostUpdate(addv=PETSc.InsertMode.ADD_VALUES, mode=PETSc.ScatterMode.REVERSE)
    set_bc(b1, bcu)
    solver1.solve(b1, u_.x.petsc_vec)
    u_.x.scatter_forward()

    # Step 2: Pressure correction step
    with b2.localForm() as loc_2:
        loc_2.set(0)
```

```
    assemble_vector(b2, L2)
    apply_lifting(b2, [a2], [bcp])
    b2.ghostUpdate(addv=PETSc.InsertMode.ADD_VALUES, mode=PETSc.ScatterMode.REVERSE)
    set_bc(b2, bcp)
    solver2.solve(b2, p_.x.petsc_vec)
    p_.x.scatter_forward()

    # Step 3: Velocity correction step
    with b3.localForm() as loc_3:
        loc_3.set(0)
    assemble_vector(b3, L3)
    b3.ghostUpdate(addv=PETSc.InsertMode.ADD_VALUES, mode=PETSc.ScatterMode.REVERSE)
    solver3.solve(b3, u_.x.petsc_vec)
    u_.x.scatter_forward()

    # Update variable with solution from this time step
    u_n.x.array[:] = u_.x.array[:]
    p_n.x.array[:] = p_.x.array[:]
b1.destroy()
b2.destroy()
b3.destroy()
solver1.destroy()
solver2.destroy()
solver3.destroy()

# 9. PyVista Visualization
topology, cell_types, geometry = vtk_mesh(V)
values = np.zeros((geometry.shape[0], 3), dtype=np.float64)

dim = mesh.geometry.dim
values[:, :dim] = u_n.x.array.real.reshape((geometry.shape[0], dim))

function_grid = pyvista.UnstructuredGrid(topology, cell_types, geometry)
function_grid["u"] = values
glyphs = function_grid.glyph(orient="u", factor=0.1) # Reduced factor slightly for Couette

tdim = mesh.topology.dim
mesh.topology.create_connectivity(tdim, tdim)
grid = pyvista.UnstructuredGrid(*vtk_mesh(mesh, tdim))

plotter = pyvista.Plotter()
plotter.add_mesh(grid, style="wireframe", color="k")
plotter.add_mesh(glyphs)
plotter.view_xy()

if not pyvista.OFF_SCREEN:
```

```
    plotter.show()
else:
    plotter.screenshot("glyphs.png")
```



6. 2D Lid Driven Cavity -

Boundary Condtions -

- Fluid Already existing inside the Cavity.
- Top Wall - Moving with unit velocity
- Left Wall - No entry flow
- Right Wall - No exit flow
- Bottom wall - No slip condition

The main characteristics of such phenomena is the formation of a vortex and two weak vortices at each bottom corners. You can run the code with parallel processing by typing in (instead of traditional - python3 script.py)-

```
mpirun -n 4 python3 script.py
```

The Fenics Code is as follows -

```
---------------------------------------------------------------------------------------
from mpi4py import MPI
from petsc4py import PETSc
import numpy as np
import pyvista

from dolfinx.fem import (
    Constant,
    Function,
    extract_function_spaces,
    functionspace,
    dirichletbc,
    form,
    locate_dofs_geometrical,
)
from dolfinx.fem.petsc import (
    assemble_matrix,
    assemble_vector,
    apply_lifting,
    create_vector,
    set_bc,
)
from dolfinx.mesh import create_unit_square
from dolfinx.plot import vtk_mesh
from basix.ufl import element
from ufl import (
    FacetNormal,
    Identity,
    TestFunction,
    TrialFunction,
    div,
    dot,
```

```
    dx,
    inner,
    lhs,
    nabla_grad,
    rhs,
    sym,
)
from dolfinx import mesh as mesh

# Define the time parameters
t = 0.0
T = 10.0
num_steps = 500
dt = T / num_steps

# Mesh (unit square, 10x10 quadrilaterals)
mesh = create_unit_square(MPI.COMM_WORLD, 50, 50, mesh.CellType.quadrilateral)

# Create function spaces
v_cg2 = element("Lagrange", mesh.basix_cell(), 2, shape=(mesh.geometry.dim,))
s_cg1 = element("Lagrange", mesh.basix_cell(), 1)
V = functionspace(mesh, v_cg2)
Q = functionspace(mesh, s_cg1)

# Trial and test functions
u = TrialFunction(V)
v = TestFunction(V)
p = TrialFunction(Q)
q = TestFunction(Q)

# --- Corrected Boundary Conditions ---

# 1. No-slip boundaries: Left, Right, and Bottom walls
def walls(x):
    return np.logical_or(np.isclose(x[1], 0.0),  # Bottom
           np.logical_or(np.isclose(x[0], 0.0),  # Left
                         np.isclose(x[0], 1.0))) # Right

bndry_walls = locate_dofs_geometrical(V, walls)
bc_walls = dirichletbc(np.zeros(mesh.geometry.dim, dtype=PETSc.ScalarType), bndry_walls, V)

# 2. Moving Lid: Top wall (excluding the exact corners to prevent singularities)
def lid(x):
    return np.logical_and(np.isclose(x[1], 1.0),
           np.logical_and(x[0] > 1e-8, x[0] < 1.0 - 1e-8))
```

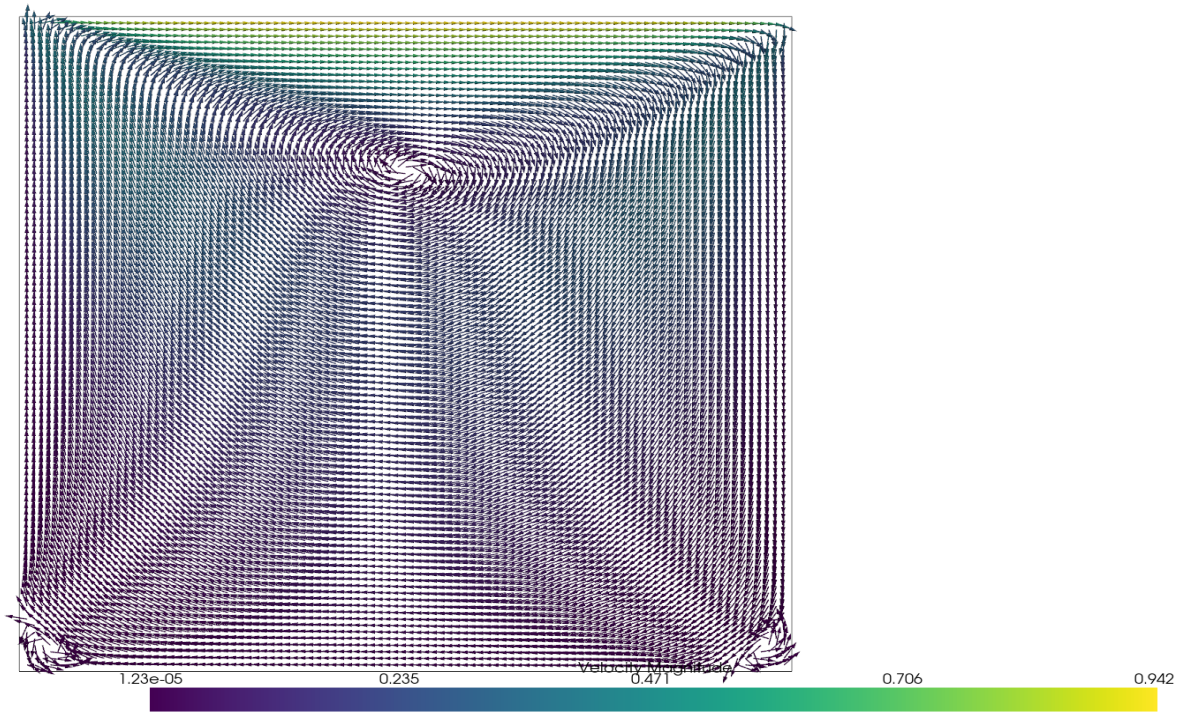
```
U_val = 1.0
u_top = np.array((U_val, 0.0), dtype=PETSc.ScalarType)
bndry_lid = locate_dofs_geometrical(V, lid)
bc_lid = dirichletbc(u_top, bndry_lid, V)

# Group velocity BCs
bcu = [bc_walls, bc_lid]

# 3. Strain-rate and Stress tensors
def epsilon(u):
    return sym(nabla_grad(u))

def sigma(u, p):
    return 2 * mu * epsilon(u) - p * Identity(mesh.geometry.dim)

# 4. Pin pressure at the absolute bottom-left corner to establish a reference point
def bottom_left(x):
    return np.logical_and(np.isclose(x[0], 0.0), np.isclose(x[1], 0.0))

bndry_p0 = locate_dofs_geometrical(Q, bottom_left)
bc_p0 = dirichletbc(PETSc.ScalarType(0.0), bndry_p0, Q)
bcp = [bc_p0]
# --- Time-dependent constants and previous solutions ---
u_n = Function(V)
u_n.name = "u_n"
p_n = Function(Q)
p_n.name = "p_n"

k = Constant(mesh, PETSc.ScalarType(dt))
mu = Constant(mesh, PETSc.ScalarType(1.0))
rho = Constant(mesh, PETSc.ScalarType(1.0))
f = Constant(mesh, PETSc.ScalarType((0.0, 0.0)))
n = FacetNormal(mesh)

U = 0.5 * (u_n + u)

# --- Step 1: Tentative velocity ---

F1 = rho * dot((u - u_n) / k, v) * dx
F1 += rho * dot(dot(u_n, nabla_grad(u_n)), v) * dx
F1 += inner(sigma(U, p_n), epsilon(v)) * dx
F1 -= dot(f, v) * dx

a1 = form(lhs(F1))
L1 = form(rhs(F1))
```

```
A1 = assemble_matrix(a1, bcs=bcu)
A1.assemble()
b1 = create_vector(extract_function_spaces(L1))

# --- Step 2: Pressure correction ---
u_ = Function(V)
u_.name = "u_"

a2 = form(dot(nabla_grad(p), nabla_grad(q)) * dx)
L2 = form(dot(nabla_grad(p_n), nabla_grad(q)) * dx - (rho / k) * div(u_) * q * dx)

A2 = assemble_matrix(a2, bcs=bcp)
A2.assemble()
b2 = create_vector(extract_function_spaces(L2))

# --- Step 3: Velocity correction ---
p_ = Function(Q)
p_.name = "p_"

a3 = form(rho * dot(u, v) * dx)
L3 = form(rho * dot(u_, v) * dx - k * dot(nabla_grad(p_ - p_n), v) * dx)

A3 = assemble_matrix(a3,bcs=bcu)
A3.assemble()
b3 = create_vector(extract_function_spaces(L3))

# --- Solvers ---
solver1 = PETSc.KSP().create(mesh.comm)
solver1.setOperators(A1)
solver1.setType(PETSc.KSP.Type.BCGS)
pc1 = solver1.getPC()
pc1.setType(PETSc.PC.Type.HYPRE)
pc1.setHYPREType("boomeramg")

solver2 = PETSc.KSP().create(mesh.comm)
solver2.setOperators(A2)
solver2.setType(PETSc.KSP.Type.CG)
pc2 = solver2.getPC()
pc2.setType(PETSc.PC.Type.HYPRE)        # Upgraded to HYPRE
pc2.setHYPREType("boomeramg")

solver3 = PETSc.KSP().create(mesh.comm)
solver3.setOperators(A3)
solver3.setType(PETSc.KSP.Type.CG)
pc3 = solver3.getPC()
pc3.setType(PETSc.PC.Type.SOR)
```

```
# --- Time loop ---
for i in range(num_steps):
    t += dt

    # Step 1: Tentative velocity
    with b1.localForm() as loc:
        loc.set(0.0)
    assemble_vector(b1, L1)
    apply_lifting(b1, [a1], [bcu])
    b1.ghostUpdate(addv=PETSc.InsertMode.ADD_VALUES, mode=PETSc.ScatterMode.REVERSE)
    set_bc(b1, bcu)
    solver1.solve(b1, u_.x.petsc_vec)
    u_.x.scatter_forward()

    # Step 2: Pressure correction
    with b2.localForm() as loc:
        loc.set(0.0)
    assemble_vector(b2, L2)
    apply_lifting(b2, [a2], [bcp])
    b2.ghostUpdate(addv=PETSc.InsertMode.ADD_VALUES, mode=PETSc.ScatterMode.REVERSE)
    set_bc(b2, bcp)
    solver2.solve(b2, p_.x.petsc_vec)
    p_.x.scatter_forward()

    # Step 3: Velocity correction
    with b3.localForm() as loc:
        loc.set(0.0)
    assemble_vector(b3, L3)
    apply_lifting(b3, [a3], [bcu])
    b3.ghostUpdate(addv=PETSc.InsertMode.ADD_VALUES, mode=PETSc.ScatterMode.REVERSE)
    set_bc(b3, bcu)
    solver3.solve(b3, u_n.x.petsc_vec)
    u_n.x.scatter_forward()

    # Update intermediate step values securely
    p_n.x.array[:] = p_.x.array[:]

# --- Cleanup ---
b1.destroy()
b2.destroy()
b3.destroy()
solver1.destroy()
solver2.destroy()
solver3.destroy()
```

```
# --- Visualization Fix: Clean Parallel Node Mapping ---
u_n.x.scatter_forward()

# 1. Get the local block/node sizes
local_nodes = V.dofmap.index_map.size_local
bs = V.dofmap.index_map_bs

# FEniCSx returns unique coordinates per node block.
dof_coords = V.tabulate_dof_coordinates()
local_dof_coords = dof_coords[:local_nodes]

# 2. Extract the local solution values and reshape them to match the nodes
u_local_flat = u_n.x.array[:local_nodes * bs].real
u_local = u_local_flat.reshape((local_nodes, bs))

u_magnitude = np.sqrt(u_local[:, 0]**2 + u_local[:, 1]**2)

# Pad to 3D so PyVista can orient the glyph arrows properly
values = np.zeros((local_nodes, 3), dtype=np.float64)
values[:, :bs] = u_local

# 3. GATHER all pieces from all processors onto Rank 0
comm = mesh.comm  # <-- Defined right here so it's accessible below!
global_coords = comm.gather(local_dof_coords, root=0)
global_vectors = comm.gather(values, root=0)
global_mags = comm.gather(u_magnitude, root=0)

# 4. Only Rank 0 builds the full plot and displays it
if comm.rank == 0:
    all_coords = np.vstack(global_coords)
    all_vectors = np.vstack(global_vectors)
    all_mags = np.concatenate(global_mags)

    # --- Filter out the boundaries entirely ---
    x_coords = all_coords[:, 0]
    y_coords = all_coords[:, 1]

    # Exclude points exactly at x=0, x=1, y=0, y=1 (with a tiny tolerance)
    tol = 1e-5
    interior_mask = (
        (x_coords > tol) & (x_coords < 1.0 - tol) &
        (y_coords > tol) & (y_coords < 1.0 - tol)
    )

    # Slice the arrays to keep only the interior data
```

```
    interior_coords = all_coords[interior_mask]
    interior_vectors = all_vectors[interior_mask]
    interior_mags = all_mags[interior_mask]
    # ------------------------------------------------

    # Create the point cloud grid using ONLY the interior points
    global_grid = pyvista.PolyData(interior_coords)
    global_grid.point_data["u_direction"] = interior_vectors
    global_grid.point_data["Velocity Magnitude"] = interior_mags

    # Generate glyphs from the filtered dataset
    glyphs = global_grid.glyph(
        orient="u_direction",
        scale=False,
        factor=0.03
    )

    # Initialize the single plotter window
    plotter = pyvista.Plotter()

    # Draw a perfect box outline based on the original limits so the frame stays intact
    full_box = pyvista.PolyData(all_coords)
    plotter.add_mesh(full_box.outline(), style="wireframe", color="k")

    # Add the pristine interior glyphs
    plotter.add_mesh(glyphs, scalars="Velocity Magnitude", cmap="viridis", show_scalar_bar=True)
    plotter.view_xy()

    if not pyvista.OFF_SCREEN:
        plotter.show()
    else:
        plotter.screenshot("couette_uniform_glyphs.png")
-------------------------------------------------------------------------------------------
```


Velocity Magnitude
1.23e-05
0.235
0.471
0.706
0.942

7. **Rayleigh-Bénard Convection -**

Boundary Condtions -

- Fluid Already existing inside the Cavity.
- Top Wall - Cold Temperature
- Left Wall - No entry flow
- Right Wall - No exit flow
- Bottom wall - Hot Temperature

The phenomena shows self organizing cyclical flow patterns. We take the Oberbeck-Boussinesq approximation which models it as a buoyancy driven flow i.e flow entirely due to different densities. The Rayleigh number has to be greater than the Critical Rayleigh number. At lower Rayleigh number, conduction dominates and we dont see any buoyancy driven flow.